

14

PyTorch

The most popular library for building neural networks

Turning deep learning ideas into real, running code

[AI/ML Engineer Learning Series · Deep Dive Edition](#)

What it is	A Python library for building and training neural networks
Made by	Meta (Facebook)
Core object	The tensor — a smart array that can use a GPU
Why loved	Feels like normal Python, easy to debug
Used by	Researchers, students, and most modern AI labs

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is PyTorch?	The library and why it's popular
2	Tensors	The core object — like NumPy with superpowers
3	Tensor Operations	Doing math with tensors
4	Autograd	How PyTorch does backprop for you
5	The GPU	Why deep learning needs it
6	Building a Model	nn.Module — the proper way
7	Loss and Optimizer	The two tools for learning
8	The Training Loop	The heart of every PyTorch program
9	Datasets and DataLoaders	Feeding data in batches
10	Saving and Loading	Keeping your trained model
11	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. What is PyTorch?

PyTorch (■■■■■■■■) is a Python library for building and training neural networks. It was made by Meta (Facebook). In the last topic you learned what a neural network IS. PyTorch is the tool you use to actually BUILD one and make it learn.

People love PyTorch for one big reason: it feels like normal Python. You can write a network step by step, print things out, and debug it just like any other code. It does the hard math of backpropagation for you automatically, so you focus on the design, not the calculus.

It's the most popular deep learning library in research and is widely used in industry too. Its main rival is TensorFlow (next topic). For learning, PyTorch is usually the friendlier start.

■ *You already used PyTorch ideas in your own projects (ResNet18, EfficientNet, fine-tuning). This guide makes the foundation rock-solid so the advanced stuff feels easy.*

2. Tensors — The Core Object

Everything in PyTorch is built on the **tensor** (■■■■■■). A tensor is just an array of numbers — exactly like a NumPy array — but with two superpowers: it can run on a GPU (very fast), and it can automatically track gradients for backpropagation.

Tensors — just arrays with any number of dimensions

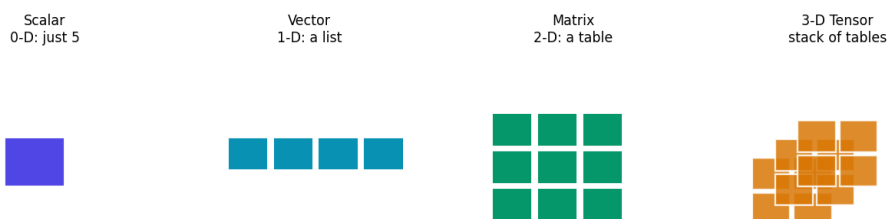


Fig 1 — Tensors can have any number of dimensions: a single number, a list, a table, or a stack of tables.

Tensor	Dimensions	Example
Scalar	0-D	a single number: 5
Vector	1-D	a list: [1, 2, 3]
Matrix	2-D	a table of rows and columns
3-D tensor	3-D	a colour image (height x width x colour)
4-D tensor	4-D	a batch of images

```
import torch

a = torch.tensor([1, 2, 3])      # a vector
b = torch.tensor([[1, 2], [3, 4]]) # a matrix

torch.zeros(2, 3)    # 2x3 of zeros
torch.ones(3, 3)     # 3x3 of ones
torch.rand(2, 2)     # random numbers 0 to 1
torch.randn(2, 2)    # random from normal distribution

a.shape      # torch.Size([3]) - the shape
a.dtype     # torch.int64      - the data type
```


■ *In a real network, autograd tracks millions of operations and computes all the gradients in one `.backward()` call. This is what makes training neural networks practical.*

5. The GPU — Why Deep Learning Needs It

A **GPU** (■■■■■■■ — graphics card) can do thousands of math operations at the same time. Neural networks are full of matrix multiplications, which GPUs are brilliant at. Training on a GPU can be 10 to 50 times faster than on a CPU.

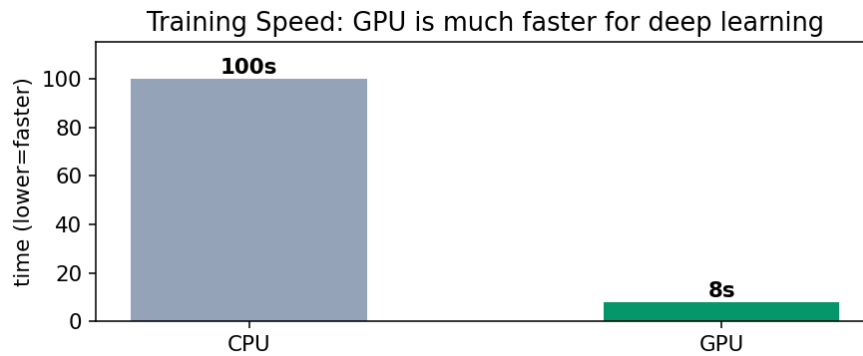


Fig 2 — The same training job finishes far faster on a GPU than a CPU.

```
# Pick GPU if available, else CPU
device = 'cuda' if torch.cuda.is_available() else 'cpu'

# Move your model and data to that device
model = model.to(device)
X = X.to(device)
y = y.to(device)
```

■ Common error: 'Expected all tensors to be on the same device'. It means your model is on the GPU but some data is still on the CPU (or the reverse). Move everything with `.to(device)`.

6. Building a Model — nn.Module

For real networks, the proper way to build a model is to make a class that inherits from **nn.Module**. You define the layers in `__init__`, and describe the forward pass in a `forward()` method.

```
import torch.nn as nn

class MyNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(4, 16)  # layer 1
        self.fc2 = nn.Linear(16, 3)  # output layer
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))  # layer 1 + activation
        x = self.fc2(x)             # output layer
        return x

model = MyNetwork()
print(model)  # shows the layers
```

■ *The forward() method is just the forward pass from the last topic, written in code. PyTorch uses it for predictions and figures out the backward pass automatically.*

7. Loss Function and Optimizer

To make a model learn, you need two more things:

- **Loss function** — measures how wrong the predictions are.
- **Optimizer** — adjusts the weights to reduce that loss (does gradient descent).

Problem type	Loss function	Optimizer (common)
Classification	<code>nn.CrossEntropyLoss()</code>	Adam
Regression	<code>nn.MSELoss()</code>	Adam or SGD
Binary yes/no	<code>nn.BCELoss()</code>	Adam

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# lr is the learning rate (step size).
# Adam is the most popular optimizer - a safe default.
```

■ Adam is the optimizer most people reach for first. A learning rate (lr) of 0.001 is a common safe starting point. If training is unstable, lower the lr; if it's too slow, raise it a little.

8. The Training Loop — The Heart of It All

The training loop is where learning happens. It's the same five steps every single time. Memorise this pattern and you can train almost anything.

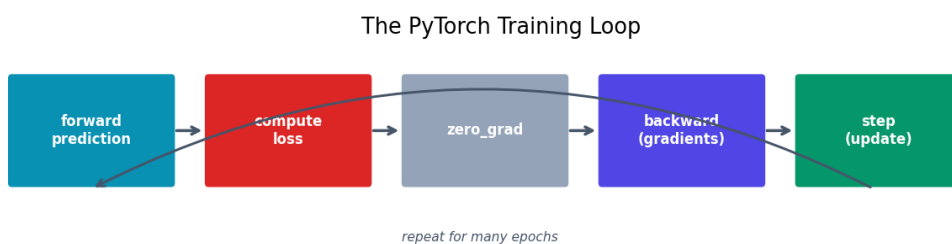


Fig 3 — The five steps of one training step, repeated for many epochs.

```
for epoch in range(100):          # repeat many times
    preds = model(X)              # 1. forward pass
    loss = loss_fn(preds, y)      # 2. measure the error

    optimizer.zero_grad()         # 3. reset old gradients
    loss.backward()               # 4. backward pass (autograd)
    optimizer.step()              # 5. update the weights

    if epoch % 10 == 0:
        print(f'Epoch {epoch}: loss = {loss.item():.4f}')
```

■ *The order matters: zero_grad BEFORE backward, and step AFTER backward. Forgetting zero_grad() is the most common PyTorch bug — gradients pile up and training breaks.*

9. Datasets and DataLoaders

Real data is too big to feed in all at once. PyTorch uses a **DataLoader** to serve the data in small **batches**, and to shuffle it. This keeps memory low and training smooth.

```
from torch.utils.data import TensorDataset, DataLoader

# Wrap your tensors in a dataset
dataset = TensorDataset(X, y)

# Serve it in batches of 32, shuffled each epoch
loader = DataLoader(dataset, batch_size=32, shuffle=True)

# Now loop over batches inside each epoch
for epoch in range(10):
    for batch_X, batch_y in loader:
        preds = model(batch_X)
        loss = loss_fn(preds, batch_y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

■ You built custom *DataLoaders* in your NMT project. The idea is always the same: the *DataLoader* hands you one batch at a time so you never load the whole dataset into memory at once.

10. Saving and Loading a Model

After training (which can take hours), you save the model so you can reuse it without training again. PyTorch saves the learned weights, called the **state_dict**.

```
# Save the trained weights
torch.save(model.state_dict(), 'model.pth')

# Later: rebuild the same model and load the weights
model = MyNetwork()
model.load_state_dict(torch.load('model.pth'))
model.eval()      # switch to evaluation mode for predictions
```

■ Always call *model.eval()* before making predictions, and *model.train()* before training. Some layers (like dropout) behave differently in each mode. Forgetting *eval()* can give slightly wrong predictions.

11. Common Mistakes

Mistake	Why it's a problem	Fix
Forgetting <code>zero_grad()</code>	Gradients add up, training breaks	Call it every step before backward
Model & data on diff devices	Crash: device mismatch	Move both with <code>.to(device)</code>
Forgetting <code>model.eval()</code>	Wrong predictions at test time	<code>eval()</code> before predicting
Wrong loss for the task	Model can't learn properly	<code>CrossEntropy=class</code> , <code>MSE=number</code>
Learning rate too high	Loss explodes or jumps	Lower lr (try 0.001)
Not moving data to GPU	Slow training	Use <code>.to(device)</code> on data too

Quick Reference Cheatsheet

Task	Code
Make a tensor	<code>torch.tensor([1,2,3])</code>
Zeros / ones	<code>torch.zeros(2,3)</code> / <code>torch.ones(2,3)</code>
Random tensor	<code>torch.randn(2,3)</code>
Shape	<code>x.shape</code>
Matrix multiply	<code>A @ B</code>
Reshape	<code>x.view(3,4)</code>
Track gradients	<code>requires_grad=True</code>
Compute gradients	<code>loss.backward()</code>
Pick device	<code>'cuda' if torch.cuda.is_available() else 'cpu'</code>
Move to device	<code>x.to(device)</code>
Define model	<code>class Net(nn.Module): ...</code>
Linear layer	<code>nn.Linear(in, out)</code>
Activation	<code>nn.ReLU()</code>
Loss (classes)	<code>nn.CrossEntropyLoss()</code>
Optimizer	<code>torch.optim.Adam(model.parameters(), lr=1e-3)</code>
Reset gradients	<code>optimizer.zero_grad()</code>
Update weights	<code>optimizer.step()</code>
Batch the data	<code>DataLoader(dataset, batch_size=32, shuffle=True)</code>
Save model	<code>torch.save(model.state_dict(), 'm.pth')</code>

Load model	<code>model.load_state_dict(torch.load('m.pth'))</code>
------------	---

The five-step loop to never forget: forward → loss → zero_grad → backward → step. That's training, every time.

Next Topic: TensorFlow — the other big deep learning library (and its friendly face, Keras). We'll compare it to PyTorch and learn when you might pick one over the other.